- KubeAgent · 项目介绍文档
  - 1. 要解决的问题
  - 2. 方案是怎么做的
    - 2.1 总体架构
    - 2.2 Harness 四原语
    - 2.3 典型闭环：一次 kubeagent fix 的完整路径
    - 2.4 关键工程决策
  - 3. 代码成果
  - 4. 用了 OpenClaw 的哪些能力

KubeAgent · 项目介绍文档

> 基于大模型 + Harness 工程方法的 Kubernetes 闭环智能运维助手

作者：white13452 · **代码仓库**：WHITE13452/kubeAgent

---

1. 要解决的问题

Kubernetes 生产环境里，SRE 每天大量时间都在做**重复、半标准化**的故障诊断 和修复：看日志、看 events、猜根因、改 YAML、删 Pod、等 Deployment 重建、再 回来确认是不是真的好了……

目前市面上的 AI 运维助手普遍存在**两类硬伤**：

1. **"Open-loop 瞎操作"**：LLM 调用写工具后就宣称"修好了"，不回头验证集群 真实状态。结果是 Pod 被删了但 Deployment 用同样的坏镜像把它拉起来， Agent 依然告诉你"任务完成"——假阳性比没干活更危险。
2. **"黑盒无审计"**：对集群做了什么改动全靠人盯日志，事后追责、复盘、合规都 拿不出结构化证据。

KubeAgent 的目标：把 Martin Fowler 提出的 **Harness 工程方法** （*Guides* + *Sensors* × *Computational* + *Inferential*）落到 Kubernetes 多 Agent 场景里，让每一次 AI 动手都有**前置护栏（Guide）+ 后置验证（Sensor）+ 结构化审计（Audit）**，从根上杜绝 open-loop 假阳性。

---

2. 方案是怎么做的

2.1 总体架构

```
┌──────────────────────────────────┐
│          CLI (Cobra)             │
│  analyze / chat / kubecheck /    │
│          fix / preflight         │
└──────────────────────────────────┘
                  │
                  ▼
     ┌──────────────────────────┐
     │        Coordinator       │
     │    Plan + DAG 执行       │
     └──────────────────────────┘
                  │          │
                  ▼          ▼
┌──────────────┐  ┌──────────────────┐
│ Diagnostician │  │    Remediator    │
│   (只读)      │  │    (写入)        │
│  Log/Event/   │  │ Human/Create/    │
│  List/Kube    │  │  Delete Tool     │
└──────────────┘  └──────────────────┘
                          │
                  ┌───────┴───────┐
                  ▼               ▼
┌──────────────────────┐  ┌──────────────┐  ┌──────────────────┐
│  Guide (前置)        │  │ Action (执行) │  │ Sensor (后置)    │
│ PreflightChain       │  │  LLM tool    │  │  K8sVerifier     │
│ - ProtectedNS        │  │    loop      │  │  轮询真实状态    │
│ - ResourceExists     │  │              │  │                  │
└──────────────────────┘  └──────────────┘  └──────────────────┘
                  │               │               │
                  └───────┬───────┴───────┘
                          ▼
                  ┌──────────────┐
                  │  AuditLogger │
                  │ Console + JSONL │
                  └──────────────┘
```

2.2 Harness 四原语

原语	实现	价值
Guide · Preflight	`PreflightChain` + `ProtectedNamespaceCheck` / `ResourceExistsCheck`	在 `kubectl create/delete` 真正发出去**之前**，拦截 `kube-system` 等受保护命名空间写操

原语	实现	价值
		作、不存在资源的删除等违规计划
Sensor · Verifier	`K8sVerifier` 轮询集群真实状态	修复动作结束后**回头校验**资源是否收敛到预期相位；open-loop 下的"假已修复"在这里被戳穿
Audit · Logger	`JSONLogAuditor` + `ConsoleReporter` + `Tee`	Preflight / Action / Verification / Decision 四类事件实时落到终端**和** JSONL，事后可机器重放、可人肉复盘
Skills · 可替换提示词	`pkg/agent/skills/*.md` + `go:embed` + `SKILLS_DIR` 热覆盖	LLM 提示词脱离代码，运行时可调，避免每次改提示词都要重新编译和发版

2.3 典型闭环：一次 `kubeagent fix` 的完整路径

以 `demo/bad-image` 场景（Deployment 引用不存在的镜像 tag）为例：

1. **Diagnostician Agent**（只读工具集：LogTool / EventTool / ListTool / KubeTool） 通过 LLM tool-use 环从 K8s API 捞 events + logs + 资源状态， 产出结构化诊断报告；
2. **Coordinator** 按 DAG 把诊断结果 → 喂给 **Remediator Agent**（写入工具集：HumanTool / CreateTool / DeleteTool，**刻意不给 KubeTool 以免 LLM 误选 `kubectl patch`**）；
3. Remediator 的每个写动作**必过 PreflightChain**：要删 `kube-system` 的 Pod? 直接 `block`；
4. Preflight 放行后，实际写 K8s 的同时把 `AuditAction` 事件落到 ConsoleReporter + JSONL；
5. 写动作结束后 **K8sVerifier** 轮询目标 Pod 状态至多 N 秒。没到 `Running` → 发出 `verification: failed`，Remediator 把任务置为 Failed，**避免了 open-loop 下的"我删了你重建了我说修好了"假阳性**；
6. 操作员通过 ConsoleReporter 彩色 `[GUIDE]/[ACTION]/[SENSOR]/[DECIDE]` 标签 实时看到进度；事后 `jq` 解析 JSONL 就能完整重放。

2.4 关键工程决策

决策	背景	效果
Remediator**不注册 KubeTool**	早期 demo 中 LLM 反复用 `kubectl patch`，但 KubeTool 是只读白名单——每次被拒 + 重试，把 10 轮 tool loop budget 打满	去掉诱惑；patch 语义通过 `CreateTool` 提交完整 YAML 实现
KubeTool 拒绝时**返回带建议的错误**	LLM 看到 `"not allowed"` 只会重试同样命令	错误消息直接点名替代工具（"use DeleteTool..."），LLM 一步转向
PreflightChain**默认 fail-closed**	策略检查本身报错时，放行比拦截更危险	检查故障等同 block，可通过 `FailClosed = false` 显式切换
独立 `kubeagent preflight` 子命令	完整 `fix` 路径要先跑 LLM 诊断，Guide 生效前可能超时/SIGKILL	纯 Guide 评估，**秒级出结果**，退出码 `0/2/3 = allow/block/warn`，适合 CI 和演示

3. 代码成果

维度	数据
新增 Go 代码	`pkg/agent/harness/` 完整子包（Verifier / Preflight / Audit / Reporter / Skills / Retry），约 1,100 行
新增子命令	`kubeagent fix`（闭环修复）+ `kubeagent preflight`（Guide 自检）
单元测试	harness + tools 包共 31 个测试用例，`go test ./...` 全绿
Skill 提示词	`diagnose.md` / `remediate.md` / `decompose.md`，`go:embed` 嵌入 + `SKILLS_DIR` 热覆盖
演示资产	`examples/demo/bad-image-deployment.yaml` + `docs/DEMO.md`（两阶段演示、8 张截图、Troubleshooting）

4. 用了 OpenClaw 的哪些能力

代码生成 & 工程落地：OpenClaw 完成了 Harness 子包（Verifier/PreflightChain/Audit/Reporter/Skills/Retry）的端到端实现，覆盖边界条件、错误处理和测试用例；同时实现 `kubeagent fix` 和 `kubeagent preflight` 两个 CLI 子命令，所有逻辑经 `go build ./...` 和 `go vet ./...` 验证通过。

多轮迭代 & 问题定位：OpenClaw 识别到 Remediator LLM 反复选择 `kubectl patch` 导致 10 轮 tool loop 耗尽的根因（KubeTool 只读白名单 + 错误消息不给出路），并提出三层联合修复方案：KubeTool 描述明确边界、拒绝错误携带替代工具建议、`skills/remediate.md` 加决策树，Claude Code 据此实现并合入 main。

文档与演示：OpenClaw 撰写了 README、docs/DEMO.md（含两阶段演示流程、8 张截图占位符、Troubleshooting 小节）以及本文档的项目介绍；其中 DEMO.md 的 screenshot 占位符已与 `docs/images/` 下的实际截图一一对应。

自动化截图执行：OpenClaw 在真实的本地 macOS 终端环境（Terminal.app，窗口 1200×800）里逐条执行完整 demo 命令流，通过 `osascript` 操作终端 + `screencapture -R` 精准截取终端区域，最终产出 8 张 PNG 截图覆盖 Step 0–5A/B 全链路，文件严格按 `00-help.png` / `01-apply.png` … `05b-audit-block.png` 命名规范存入 `docs/images/`。

Git 工作流协作：OpenClaw 按需创建 `claude/fix-remediator-tool-selection` 分支，将代码修复与文档更新一同 commit 后推送远端，并通过 GitHub PR 描述和 review checklist 与 maintainer（怀特）协作，最终合入 main。